

Week 9: Concurrency Control Techniques

4.3. Multi-version Concurrency Control

Objective: Understand the principles and implementation of Multi-version Concurrency Control (MVCC).

Lab Activities:

1. Introduction to MVCC:

- Overview of MVCC concepts.
- Differences between MVCC and other concurrency control mechanisms.

2. Setup and Configuration:

- Setting up a SQL Server environment.
- Enabling snapshot isolation level for MVCC.

3. Hands-on with MVCC:

- **Create Sample Database and Tables:**

```
CREATE DATABASE mvcc_lab;
USE mvcc_lab;
CREATE TABLE accounts (
    id INT PRIMARY KEY IDENTITY(1,1),
    name VARCHAR(100),
    balance DECIMAL(10, 2)
);
INSERT INTO accounts (name, balance) VALUES ('Alice', 1000), ('Bob', 1000);
```

- **Enable Snapshot Isolation:**

```
ALTER DATABASE mvcc_lab SET ALLOW_SNAPSHOT_ISOLATION ON;
ALTER DATABASE mvcc_lab SET READ_COMMITTED_SNAPSHOT ON;
```

- **Perform Concurrent Transactions:**

```
-- Transaction 1
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRAN;
SELECT * FROM accounts WHERE name = 'Alice';
UPDATE accounts SET balance = balance - 100 WHERE name = 'Alice';
```

```

-- Transaction 2
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRAN;
SELECT * FROM accounts WHERE name = 'Alice';
UPDATE accounts SET balance = balance + 100 WHERE name = 'Alice';

-- Commit Transaction 1
COMMIT;

-- Commit Transaction 2
COMMIT;

```

○ **Analyze Results:**

```
SELECT * FROM accounts;
```

Analysis

- MVCC ensures that each transaction works with a snapshot of the data at the start of the transaction.
- The updates in both transactions are isolated from each other, preventing conflicts.
- The final state of Alice's balance will reflect both updates (one subtracting and one adding 100).

Expected Output

After committing both transactions, the expected output will show Alice's balance correctly reflecting both updates:

```

id | name  | balance
---|-----|-----
1  | Alice | 1000.00
2  | Bob   | 1000.00

```

4. Conflict Resolution:

- Experiment with scenarios that lead to conflicts and observe how MVCC resolves them.

5. Case Study:

- Review a real-world application using MVCC and analyze its benefits.

4.4. Locks for Indexes

Objective: Learn about locking mechanisms for indexes and their importance in database operations.

Lab Activities:

1. Introduction to Index Locks:

- Overview of index locking and its types (e.g., shared, exclusive).

2. Setup and Configuration:

- Setting up a SQL Server environment to experiment with index locks.

3. Hands-on with Index Locks:

- **Create Indexes:**

```
CREATE INDEX idx_name ON accounts (name);
```

- **Perform Operations and Monitor Locks:**

```
-- First session
BEGIN TRAN;
SELECT * FROM accounts WITH (UPDLOCK) WHERE name = 'Alice';
-- View locks
SELECT * FROM sys.dm_tran_locks WHERE resource_associated_entity_id =
OBJECT_ID('accounts');

-- Second session
BEGIN TRAN;
SELECT * FROM accounts WITH (UPDLOCK) WHERE name = 'Alice';

-- This will wait until the first transaction is committed
```

4. Analyzing Lock Contention:

- Simulate high lock contention scenarios.
- Measure the impact on performance and explore ways to mitigate contention.

5. Best Practices:

- Discuss best practices for managing index locks in production environments.

Week 10: Database Recovery Techniques

5.1. Backup and Recovery Concepts

Objective:

- Gain a thorough understanding of backup and recovery concepts.

Lab Activities:

1. Introduction to Backup and Recovery:

- Overview of different types of backups (e.g., full, incremental, differential).

2. Backup Strategies:

- Discuss various backup strategies and their use cases.

3. Hands-on Backup:

- **Perform Full and Incremental Backups:**

```
-- Full backup
```

```
BACKUP DATABASE mvcc_lab TO DISK = 'C:\backups\mvcc_lab_full.bak';
```

```
-- Differential backup
```

```
BACKUP DATABASE mvcc_lab TO DISK = 'C:\backups\mvcc_lab_diff.bak' WITH  
DIFFERENTIAL;
```

- **Verify Backup Integrity:**

```
RESTORE VERIFYONLY FROM DISK = 'C:\backups\mvcc_lab_full.bak';
```

5.2. Backup

Objective: Implement and manage database backups effectively.

Lab Activities:

1. Automating Backups:

- **Automated Backup Script:**

```
-- Full backup script
DECLARE @BackupFile NVARCHAR(255) = 'C:\backups\mvcc_lab_full_' +
    CONVERT(VARCHAR, GETDATE(), 112) + '.bak';

BACKUP DATABASE mvcc_lab TO DISK = @BackupFile;
```

- **Schedule Backups using SQL Server Agent:**

```
-- Create a SQL Server Agent job for automated backups
-- Schedule the job to run daily at 2 AM
```

2. Storing Backups:

- Explore and configure storage options (e.g., local, cloud).

3. Backup Best Practices:

- Discuss and implement best practices for backup maintenance.

5.3. Recovery

Objective: Learn techniques for recovering databases from different types of failures.

Lab Activities:

1. Introduction to Recovery Techniques:

- Overview of recovery processes for different failure scenarios.

2. Hands-on Recovery:

- **Simulate Database Failures:**

```
DROP TABLE accounts;
```

- **Perform Recovery:**

```
RESTORE DATABASE mvcc_lab FROM DISK = 'C:\backups\mvcc_lab_full.bak';
```

- **Validate Recovered Data:**

```
SELECT * FROM accounts;
```

3. **Point-in-Time Recovery:**

- **Implement PITR:**

```
-- Assuming transaction log backups are available  
RESTORE DATABASE mvcc_lab FROM DISK = 'C:\backups\mvcc_lab_full.bak' WITH  
NORECOVERY;  
RESTORE LOG mvcc_lab FROM DISK = 'C:\backups\mvcc_lab_log.bak' WITH STOPAT =  
'2024-12-22 10:00:00', RECOVERY;
```

4. **Disaster Recovery Plan:**

- Develop and test a disaster recovery plan for a sample organization.